

ID Expert

Априорные решения машин

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Функциональная спецификация и требования к разработке

Универсальная платформа Телематики для разнородного парка техники

Подготовила: Бужор Александра

Системный / бизнес-аналитик проекта

Версия 1.0 • 08.07.2026 • на согласование заказчику

г. Санкт-Петербург

Оглавление

Аннотация и история версий	4
1. Общие сведения	5
1.1. Назначение и область применения.....	5
1.2. Цель работы	5
Требования к системе (обзор)	5
1.3. Термины, определения и сокращения	5
2. Анализ требований	6
2.1. Роли пользователей.....	6
2.2. Бизнес-требования к платформе	6
2.3. Перечень телеметрических параметров	6
2.3.1. Обязательные для всех типов техники	6
2.3.2. Для ДВС-техники (дизельные тракторы, погрузчики)	7
2.3.3. Для электрической техники (EV).....	7
2.3.4. Для ВАТС (роботы / автономная техника).....	7
2.3.5. Специфичные для отдельных ТС	8
2.4. Каталог событий.....	8
2.5. Границы и допустимые значения показателей.....	8
3. Пользовательские сценарии (User Stories)	9
3.1. Методика приоритизации.....	9
3.2. Матрица сценариев по группам	9
3.3. Детальные User Stories	9
4. Функциональные требования	11
5. Нефункциональные требования.....	11
6. Архитектура сбора и хранения данных.....	12
6.1. Общая схема	12
6.2. Путь данных «от ТС до платформы».....	12
6.3. Протоколы и каналы связи.....	12
6.4. Обработка офлайн-режима и достоверность данных	13
6.5. Слои хранения (hot / warm / cold)	13
7. Универсальная модель данных	13
7.1. Принципы проектирования	13
7.2. Описание ключевых сущностей.....	13
7.3. ER-диаграмма	14
7.4. Партиционирование и индексы	14
Фрагмент DDL: таблица метрик	14
8. Спецификация API	15
8.1. Общие принципы	15
8.2. Основные методы	15
8.3. Форматы входных / выходных данных	15
Вход: батч телеметрии (POST /ingest/telemetry)	15
Выход: временной ряд (GET /vehicles/FRK-014/metrics).....	16
Выход: состояние парка (GET /fleet/state)	16

9. Прототипы пользовательских интерфейсов (Wireframes).....	16
9.1. Мониторинг парка — карта в реальном времени	16
9.2. Карточка ТС — телеметрия в реальном времени	16
9.3. Аналитика нарушений — дашборд	17
9.4. Отчёт по эффективности + сравнение ТС	18
10. Требования к инфраструктуре и DevOps.....	18
10.1. Развёртывание	18
10.2. CI/CD	18
10.3. Наблюдаемость и эксплуатация	18
11. Этапы реализации.....	19
12. Ожидаемые результаты и приложения	19

Аннотация и история версий

Настоящий документ — техническое задание (ТЗ) и функциональная спецификация на проектирование и разработку **универсальной платформы Телематики** — системы сбора, хранения, обработки и предоставления данных о работе разнородного парка ТС: тракторов (ДВС), электрических вилочных погрузчиков, тележек и роботов (BATS). Документ адресован команде реализации: backend-разработчикам, специалистам по БД, DevOps-инженерам, frontend-команде и владельцу продукта.

Документ охватывает: роли пользователей и бизнес-требования; полный перечень телеметрических параметров и событий с границами значений; пользовательские сценарии (User Stories) с приоритизацией; функциональные и нефункциональные требования; архитектуру сбора и хранения телеметрии; универсальную расширяемую модель данных с ER-диаграммой; спецификацию REST API (OpenAPI); форматы входных/выходных данных; прототипы интерфейсов; требования к инфраструктуре и план поэтапной реализации.

Версия	Дата	Автор	Описание
0.1	01.07.2026	А. Бужор	Черновик: структура, анализ требований
0.5	05.07.2026	А. Бужор	Модель данных, API, прототипы интерфейсов
1.0	08.07.2026	А. Бужор	Полная редакция для согласования с заказчиком

1. Общие сведения

1.1. Назначение и область применения

Платформа предназначена для централизованного мониторинга и анализа работы разнородного парка техники вне зависимости от производителя, типа привода и степени автоматизации. Область применения — логистические комплексы, склады, производственные площадки, агропредприятия и любые организации, эксплуатирующие смешанный парк самоходной и несамоходной техники.

Ключевая идея проекта — универсальность. Платформа одинаково работает с дизельным трактором, электрическим погрузчиком и автономным роботом, приводя их разнородную телеметрию к единым форматам данных и единому API. Добавление нового типа техники или нового параметра телеметрии не должно требовать изменения структуры базы данных и переписывания API.

1.2. Цель работы

Разработать детальное ТЗ, достаточное для начала реализации без дополнительного анализа: пользовательские сценарии, логику работы системы, интерфейсы взаимодействия между модулями, прототипы интерфейсов, модель данных, архитектуру сбора и хранения телеметрии, а также API для приёма и выдачи данных.

Требования к системе (обзор)

Требование	Содержание
Поддержка разных ТС	Тракторы (ДВС), погрузчики (электро), тележки, роботы (BATS) и другие типы в рамках единой модели.
Расширяемость	Добавление нового типа техники или параметра телеметрии без миграции схемы БД и без изменения контракта API.
Масштабируемость	Парк от 1 до 1000+ ТС; горизонтальное масштабирование приёма и обработки; готовность к росту частоты и объёма данных.
Универсальность	Единые форматы данных и единый REST API для всех типов техники.

1.3. Термины, определения и сокращения

Термин	Определение
Телематика / телеметрия	Дистанционный сбор и передача данных о состоянии и работе ТС.
ТС	Транспортное средство (единица техники).
ДВС	Двигатель внутреннего сгорания (дизель/бензин).
EV	Электрическая техника (Electric Vehicle).
BATS	Высокоавтоматизированное ТС (робот, AGV/AMR), выполняющее миссии автономно.
TBU	Телематический блок — бортовое устройство сбора и передачи телеметрии.
Метрика (Metric)	Значение одного датчика в конкретный момент времени.
Событие (Event)	Зафиксированный факт нарушения/состояния (перегрев, выход из геозоны и т. п.).
Миссия (Mission)	Задание для BATS (маршрут/действия) с отслеживанием прогресса.
Геозона (Geofence)	Виртуальная граница на карте; вход/выход генерирует событие.
JSONB	Бинарный JSON-тип PostgreSQL для хранения вариативных полей.
MQTT	Лёгкий протокол обмена сообщениями для IoT-устройств.
RBAC	Ролевая модель управления доступом.
SoC / SoH	Уровень заряда / состояние здоровья батареи.
OBD-II / J1939	Диагностические протоколы бортовых сетей ТС.
KPI	Ключевой показатель эффективности.

2. Анализ требований

2.1. Роли пользователей

Выделены следующие роли; доступ разграничивается по модели RBAC. Одному сотруднику может быть назначено несколько ролей.

Роль	Основные задачи	Ключевые разделы
Диспетчер / оператор мониторинга	Наблюдение за парком в реальном времени, реакция на тревоги, координация.	Карта, телеметрия, события
Аналитик парка	Анализ нарушений и эффективности, сравнение ТС, отчёты.	Дашборды, отчёты, сравнение
Технический специалист / механик	Контроль техсостояния, планирование ТО по моточасам/пробегу.	Телеметрия, события, ТО
Руководитель / менеджер парка	Контроль KPI, отчётность по эффективности и затратам.	Отчёты, сводки
Оператор BATC	Постановка и контроль миссий, обработка вмешательств и аварийных остановов.	Миссии, телеметрия BATC
Администратор системы	Регистрация ТС и датчиков, справочники, пользователи, правила алертов.	Администрирование
Внешняя система / интегратор	Программный доступ к данным через API (ERP, BI, MES).	REST API

2.2. Бизнес-требования к платформе

BR-1. Единое окно мониторинга всего разнородного парка в реальном времени на карте.

BR-2. Снижение простоев и рост коэффициента использования техники за счёт оперативных данных и аналитики.

BR-3. Раннее выявление технических проблем (перегрев, неисправности, низкий заряд/топливо) для предотвращения простоев и аварий.

BR-4. Контроль дисциплины эксплуатации: превышение скорости, простои, выход из разрешённых зон.

BR-5. Прозрачная отчётность по эффективности, расходу топлива/энергии и затратам для управленческих решений.

BR-6. Поддержка автономной техники (BATC): постановка миссий, контроль автономности и безопасности.

BR-7. Быстрое подключение новой техники и новых параметров без доработки ядра системы.

BR-8. Интеграция с внешними системами (ERP/BI/MES) через открытый API.

BR-9. Работа в условиях нестабильной связи: буферизация на устройстве и достоверность исторических данных.

2.3. Перечень телеметрических параметров

Параметры сгруппированы по применимости. Каждый параметр описывается записью справочника **sensor_type** (код, единица, тип значения, границы, категория). Коды датчиков (например, **battery.temp**) сквозные — они одинаковы в API, модели данных и на устройстве.

2.3.1. Обязательные для всех типов техники

Код датчика	Параметр	Ед.	Тип
gps.location	Геопозиция (широта, долгота, скорость, курс, спутники, HDOP)	—	json
speed	Скорость	км/ч	numeric
heading	Курс (направление движения)	град	numeric

Код датчика	Параметр	Ед.	Тип
odometer	Пробег / наработка	км	numeric
status	Статус ТС (в работе / простой / выключен / движение / ошибка)	—	text
ignition	Зажигание / питание	on/off	boolean
signal.rssi	Уровень сигнала связи	dBm	numeric
power.supply_voltage	Напряжение бортовой сети	В	numeric
gps.quality	Качество GPS (спутники, HDOP)	—	json

2.3.2. Для ДВС-техники (дизельные тракторы, погрузчики)

Код датчика	Параметр	Ед.	Тип
engine.rpm	Обороты двигателя	об/мин	numeric
fuel.level	Уровень топлива	%	numeric
fuel.rate	Мгновенный расход топлива	л/ч	numeric
coolant.temp	Температура охлаждающей жидкости	°С	numeric
oil.pressure	Давление масла	кПа	numeric
oil.temp	Температура масла	°С	numeric
engine.hours	Моточасы двигателя	ч	numeric
engine.load	Нагрузка на двигатель	%	numeric
adblue.level	Уровень AdBlue/DEF (SCR)	%	numeric
battery.voltage_start	Напряжение пусковой АКБ	В	numeric
dtc.codes	Коды неисправностей (OBD-II/J1939)	—	json

2.3.3. Для электрической техники (EV)

Код датчика	Параметр	Ед.	Тип
battery.soc	Уровень заряда (State of Charge)	%	numeric
battery.soh	Состояние здоровья батареи	%	numeric
battery.voltage	Напряжение тяговой батареи	В	numeric
battery.current	Ток заряда/разряда	А	numeric
battery.temp	Температура батареи/ячеек	°С	numeric
battery.cycles	Количество циклов заряда	шт	numeric
charging.state	Статус зарядки (заряд/разряд/простой)	—	text
range.remaining	Остаточный запас хода / время работы	мин	numeric
motor.temp	Температура двигателя/инвертора	°С	numeric
power.consumption	Мгновенная мощность потребления	кВт	numeric

2.3.4. Для ВАС (роботы / автономная техника)

Код датчика	Параметр	Ед.	Тип
autonomy.mode	Режим (ручной / автономный / дистанционный / аварийный)	—	text
mission.id	Идентификатор текущей миссии	—	text
mission.progress	Прогресс миссии	%	numeric
localization.accuracy	Точность позиционирования	см	numeric
estop.state	Состояние аварийной кнопки (E-Stop)	—	boolean
obstacle.status	Статус системы обнаружения препятствий	—	text
interventions.count	Количество вмешательств оператора	шт	numeric
payload.loaded	Наличие груза	—	boolean

Код датчика	Параметр	Ед.	Тип
nav.status	Состояние системы навигации	—	text

2.3.5. Специфичные для отдельных ТС

ТС	Код датчика	Параметр
Погрузчик	lift.height / lift.load_weight / mast.tilt / hydraulics.pressure	Высота подъёма ви́л, вес груза, наклон мачты, давление гидравлики
Трактор	pto.rpm / implement.position / hitch.state / hydraulics.pressure	Обороты ВОМ, положение навесного, состояние сцепки, давление гидросистемы
Тележка	cargo.weight / brake.state	Вес груза, состояние тормозов
Робот-уборщик	tank.level / brush.state	Уровень воды/раствора, состояние щёток

Список не является закрытым. Новый параметр добавляется записью в справочник sensor_type и сразу доступен в приёме, хранении и выдаче — без изменения схемы БД и контракта API. В этом состоит расширяемость модели.

2.4. Каталог событий

События фиксируются либо на устройстве (edge-детекция), либо в потоковой обработке платформы по правилам алертов. Каждое событие имеет тип, важность (info / warning / critical), метку времени, геопозицию и payload с деталями.

Код события	Событие	Важность
overspeed	Превышение скорости	warning
geofence_exit / geofence_enter	Выход / вход в геозону	warning / info
harsh_braking / harsh_accel	Резкое торможение / ускорение	info
excessive_idling	Простой сверх нормы	info
low_fuel / low_battery	Низкий уровень топлива / заряда	warning
overheat	Перегрев (двигатель/батарея)	critical
fault_dtc	Неисправность (код DTC)	warning/critical
impact	Удар / столкновение (по акселерометру)	critical
tilt	Опасный крен / риск опрокидывания	critical
estop_triggered	Аварийный останов ВАТС (E-Stop)	critical
operator_intervention	Вмешательство оператора в автономный режим	info
mission_start / mission_end	Начало / завершение миссии	info
charging_start / charging_end	Начало / завершение зарядки	info
shift_start / shift_end	Начало / завершение смены	info
connection_lost / connection_restored	Потеря / восстановление связи	warning / info
unauthorized_use	Использование вне графика	warning
maintenance_due	Подошёл срок ТО (по моточасам/пробегу)	info

2.5. Границы и допустимые значения показателей

Для валидации входных данных и генерации событий заданы физические границы (для отбраковки заведомо ошибочных значений) и пороги тревог (настраиваемые в правилах алертов). Значения ниже — базовые; уточняются по паспортам конкретных моделей.

Параметр	Ед.	Физ. диапазон	Норма	Порог тревоги
speed	км/ч	0...120	по регламенту зоны	> лимита зоны
engine.rpm	об/мин	0...4000	600...2200	> 2500 длительно
fuel.level	%	0...100	> 20	< 10

Параметр	Ед.	Физ. диапазон	Норма	Порог тревоги
coolant.temp	°C	-40...130	80...95	> 105
oil.pressure	кПа	0...700	150...500	< 100 на оборотах
battery.soc	%	0...100	> 30	< 15
battery.temp	°C	-40...120	15...45	> 55
battery.voltage	В	0...1000	по модели	вне окна $\pm 10\%$
localization.accuracy	см	0...1000	< 10	> 30 (для BATC)
estop.state	—	true/false	false	true

3. Пользовательские сценарии (User Stories)

3.1. Методика приоритизации

Приоритет задаётся по методике **MoSCoW**: Must (обязательно, MVP), Should (желательно), Could (дополнительно), Won't (вне текущего объёма). Каждая история имеет идентификатор US-N, роль, ценность и критерии приёмки (в нотации Given/When/Then).

3.2. Матрица сценариев по группам

Группа	Пример сценария	Приоритет
Мониторинг парка	Пользователь видит на карте все машины с текущим статусом.	Must
Просмотр телеметрии	Пользователь выбирает машину и видит её параметры в реальном времени.	Must
Аналитика нарушений	Пользователь видит дашборд с нарушениями за период.	Must
Оповещения	Пользователь получает push/email при критическом событии.	Must
Отчёты	Пользователь формирует отчёт по эффективности парка за месяц.	Should
Управление BATC	Оператор ставит миссию роботу и следит за прогрессом.	Should
Сравнение машин	Пользователь сравнивает показатели двух ТС.	Could
Администрирование	Администратор регистрирует новое ТС и его датчики.	Must
Интеграция	Внешняя система забирает данные через API.	Should

3.3. Детальные User Stories

US-1. Как **Диспетчер**, я хочу видеть все ТС на карте с цветовым статусом, чтобы оперативно оценивать обстановку по всему парку. **[Must]**

Критерии приёмки:

- Дана карта парка — когда открыт экран мониторинга, то отображаются все активные ТС маркерами с цветом по статусу (в работе/простой/тревога/нет связи).
- Когда состояние ТС меняется, то маркер обновляется не позднее чем через 5 секунд.
- Когда пользователь наводит на маркер, то показывается всплывающая карточка (ID, тип, статус, ключевые метрики).

US-2. Как **Диспетчер**, я хочу открыть карточку конкретного ТС и видеть телеметрию в реальном времени, чтобы диагностировать состояние машины. **[Must]**

Критерии приёмки:

- Когда выбран ТС, то отображаются текущие значения его датчиков и графики за последний час.
- Когда приходит новое значение, то показатели и графики обновляются в реальном времени (WebSocket/SSE).
- Когда датчик выходит за порог, то значение подсвечивается и появляется отметка события.

US-3. Как **Аналитик**, я хочу видеть дашборд нарушений за выбранный период с фильтрами, чтобы выявлять проблемные ТС и типы нарушений. **[Must]**

Критерии приёмки:

- Когда задан период и фильтры (тип ТС, тип нарушения, площадка), то показываются KPI, динамика по дням и разбивка по типам.
- Когда выбран элемент диаграммы, то реестр нарушений фильтруется соответственно.
- Когда данных нет, то отображается пустое состояние с пояснением.

US-4. Как **Любой пользователь**, я хочу получать оповещения о критических событиях, чтобы реагировать без постоянного наблюдения за экраном. **[Must]**

Критерии приёмки:

- Когда возникает событие severity=critical, то подписанные пользователи получают push и/или email в течение 30 секунд.
- Когда пользователь квитирует событие, то оно помечается обработанным и повторные оповещения по нему прекращаются.

US-5. Как **Руководитель**, я хочу формировать отчёт по эффективности парка за месяц, чтобы принимать управленческие решения. **[Should]**

Критерии приёмки:

- Когда выбран шаблон и период, то формируется отчёт с коэффициентами использования и готовности, расходом ресурсов и простоями.
- Когда нажата выгрузка, то отчёт экспортируется в PDF/XLSX.
- Когда настроено расписание, то отчёт присылается на email автоматически.

US-6. Как **Оператор BATC**, я хочу поставить роботу миссию и отслеживать её выполнение, чтобы управлять автономной техникой. **[Should]**

Критерии приёмки:

- Когда создана миссия с маршрутом, то она передаётся BATC и переходит в статус active.
- Когда робот выполняет миссию, то прогресс и позиция обновляются в реальном времени.
- Когда происходит вмешательство/аварийный останов, то фиксируется событие и оператор уведомляется.

US-7. Как **Аналитик**, я хочу сравнить два ТС по ключевым показателям, чтобы выявлять отстающую технику и обучать операторов. **[Could]**

Критерии приёмки:

- Когда выбраны два ТС и период, то показывается таблица сравнения (наработка, использование, удельный расход, нарушения) с автоматическим выводом.

US-8. Как **Администратор**, я хочу зарегистрировать новое ТС и его набор датчиков, чтобы быстро подключать технику к платформе. **[Must]**

Критерии приёмки:

- Когда заданы тип ТС и параметры, то ТС создаётся и появляется в реестре и на карте после первого пакета телеметрии.
- Когда добавляется новый тип датчика, отсутствующий в справочнике, то он создаётся без изменения схемы БД.

US-9. Как **Внешняя система**, я хочу получать телеметрию и события через API по ключу, чтобы интегрировать данные в ERP/BI. **[Should]**

Критерии приёмки:

- Когда выполнен запрос с валидным API-ключом, то возвращаются данные в JSON согласно OpenAPI.
- Когда превышен лимит запросов, то возвращается 429 с заголовками ограничения.

4. Функциональные требования

Функциональные требования (FR) детализируют поведение системы и трассируются к бизнес-требованиям (BR) и историям (US).

ID	Требование	Связь
FR-01	Приём телеметрии от устройств батчами по MQTT и HTTP, с аутентификацией устройства и дедупликацией.	BR-1, US-1
FR-02	Буферизация на устройстве при потере связи и досыл данных с исходными метками времени.	BR-9
FR-03	Валидация значений по границам sensor_type; отбраковка/пометка недостоверных.	BR-3
FR-04	Нормализация единиц измерения к каноническим при приёме.	BR-7
FR-05	Хранение временных рядов метрик с партиционированием по времени.	BR-2
FR-06	Детекция событий по настраиваемым правилам (пороги, геозоны, длительность).	BR-3, BR-4
FR-07	Поддержка текущего состояния каждого ТС (last-known) для карты.	BR-1, US-1
FR-08	Отображение парка на карте с фильтрами и обновлением в реальном времени.	US-1
FR-09	Карточка ТС: текущие значения, графики, журнал событий, трек.	US-2
FR-10	Дашборд нарушений с фильтрами, диаграммами и реестром.	US-3
FR-11	Оповещения (push/email/webhook) с квитиованием.	US-4
FR-12	Отчёты по эффективности с экспортом (PDF/XLSX) и расписанием.	US-5
FR-13	Управление миссиями BATC (создание, статусы, прогресс).	US-6
FR-14	Сравнение двух ТС по показателям за период.	US-7
FR-15	Реестр ТС и датчиков, справочники, регистрация без миграций.	US-8, BR-7
FR-16	Управление геозонами (CRUD, полигоны).	BR-4
FR-17	Управление правилами алертов (условия, действия).	BR-3
FR-18	REST API для приёма и выдачи данных (OpenAPI).	US-9, BR-8
FR-19	RBAC: разграничение доступа по ролям, аудит действий.	все

5. Нефункциональные требования

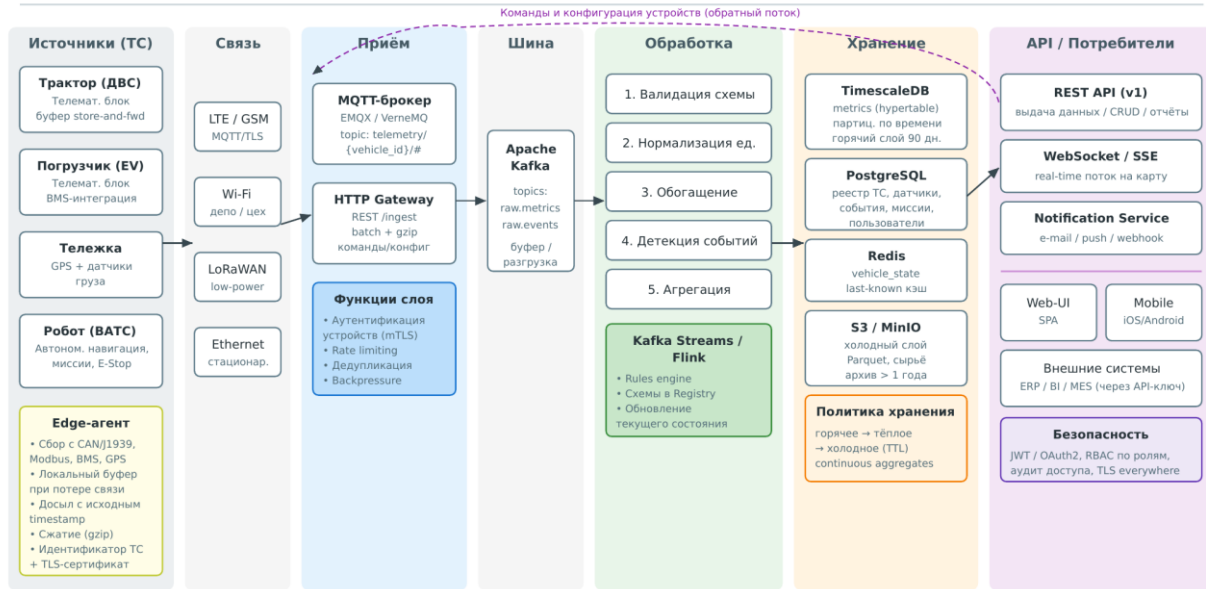
Категория	Требование
Производительность	Приём $\geq 5\,000$ сообщений/с при 1000+ ТС; задержка «устройство → карта» ≤ 5 с (p95); ответ query-API ≤ 300 мс (p95) на типовых выборках.
Масштабируемость	Горизонтальное масштабирование слоёв приёма и обработки (stateless-сервисы + Kafka); секционирование метрик; шардирование при росте объёма.
Надёжность / доступность	Доступность $\geq 99,5\%$ в год; отсутствие потерь принятой телеметрии (at-least-once + дедупликация); репликация БД, резервное копирование.
Целостность данных	Идемпотентность приёма по (vehicle_id, sensor, ts); контроль последовательности батчей (seq).
Безопасность	TLS для всех каналов; mTLS/токены для устройств; JWT/OAuth2 и RBAC для пользователей; аудит доступа; шифрование секретов.
Хранение / ретенция	Горячий слой (оперативный) — 90 дней; тёплый (агрегаты) — 1 год; холодный (Parquet в S3) — по политике заказчика.
Наблюдаемость	Централизованные логи, метрики (Prometheus), трассировка, алерты по здоровью платформы.
Совместимость / открытость	Версионизируемый REST API (OpenAPI); стандартные форматы (JSON, GeoJSON); экспорт CSV/XLSX.
Локализация	Интерфейс на русском; поддержка часовых поясов; хранение времени в UTC.
Приватность / соответствие	Разграничение данных по организациям/площадкам; соответствие внутренним политикам заказчика по ПДн.

6. Архитектура сбора и хранения данных

6.1. Общая схема

Архитектура — слоистая и событийно-ориентированная. Телеметрия движется «снизу вверх»: от бортового устройства через слой приёма и шину сообщений в потоковую обработку, затем в хранилища и к API. Обратный поток (команды и конфигурация) идёт «сверху вниз» к устройствам.

Архитектура сбора, обработки и хранения телеметрии



Разработчик системного проекта: Бужор Александра. Проект: универсальная платформа Телематтики, «ID Expert».

Рис. 1. Архитектура сбора, обработки и хранения телеметрии.

6.2. Путь данных «от ТС до платформы»

- Сбор на борту.** Телематический блок (TBU) собирает данные с бортовых шин (CAN/J1939, Modbus), BMS (для EV), GPS и специфических датчиков; формирует унифицированные метрики с метками времени.
- Буферизация.** При потере связи данные сохраняются в локальном буфере (store-and-forward) и досылаются при восстановлении с исходными ts.
- Передача.** Основной канал — MQTT поверх TLS (topic telemetry/{vehicle_id}/...); альтернатива — HTTP-батчи с gzip. Для команд — отдельный обратный канал.
- Приём.** Слой приёма (MQTT-брокер + HTTP Gateway) аутентифицирует устройство (mTLS/токен), применяет rate limiting и дедупликацию, публикует в шину.
- Шина.** Apache Kafka буферизует поток и разделяет его на топики (raw.metrics, raw.events), сглаживая пики и разгружая обработку.
- Обработка.** Поточный процессор (Kafka Streams/Flink): валидация схемы → нормализация единиц → обогащение (справочные данные) → детекция событий → агрегация; обновляет текущее состояние ТС.
- Хранение.** Метрики — в TimescaleDB (партиции по времени); реестр/события/миссии — в PostgreSQL; текущее состояние — в Redis; сырьё и архив — в объектном хранилище (Parquet).
- Выдача.** REST API отдаёт данные потребителям; WebSocket/SSE транслирует real-time на карту; Notification Service рассылает оповещения.

6.3. Протоколы и каналы связи

Протокол	Назначение	Примечание
MQTT / TLS	Основной приём телеметрии от подвижной техники	Экономичен на мобильных каналах, QoS 1

Протокол	Назначение	Примечание
HTTPS (REST)	Батч-приём, команды, конфигурация, стационарные ТС	gzip, идемпотентность по seq
WebSocket / SSE	Трансляция real-time потребителям (карта)	Подписка по ТС/площадке
LoRaWAN / CoAP	Низкоскоростная телеметрия маломощных устройств	Опционально, редкие пакеты

6.4. Обработка офлайн-режима и достоверность данных

- Устройство буферизует данные при отсутствии связи и досылает их пакетами, сохраняя исходные метки времени события (event time), а не времени приёма.
- Платформа различает event time (когда произошло) и received_at (когда получено), что обеспечивает корректную хронологию исторических данных.
- Дедупликация выполняется по ключу (vehicle_id, sensor, ts); повторно присланные точки не создают дублей.
- Контроль потерь — по сквозному номеру батча (seq); пропуски выявляются и логируются.

6.5. Слои хранения (hot / warm / cold)

Слой	Что хранит	Технология	Срок
Горячий	Сырые метрики высокой частоты, оперативные запросы	TimescaleDB (партиции)	~90 дней
Тёплый	Почасовые/посменные агрегаты для отчётов и графиков	Continuous aggregates	~1 год
Холодный	Архив сырья для аудита и переобработки	S3/MinIO, Parquet	по политике
Оперативный кэш	Текущее состояние парка (для карты)	Redis + vehicle_state	актуальное

7. Универсальная модель данных

7.1. Принципы проектирования

- **Справочники вместо жёсткой схемы.** Типы техники, датчиков и событий вынесены в справочники (vehicle_type, sensor_type, event_type). Новый тип — это новая строка справочника.
- **«Длинная» таблица метрик.** Телеметрия хранится как временной ряд «одна строка = одно измерение одного датчика». Добавление параметра не требует ALTER TABLE.
- **JSONB для вариативности.** Вендорные и составные атрибуты (attributes, payload, value_json) хранятся в JSONB с GIN-индексами.
- **Партиционирование по времени.** Таблицы metric и event секционируются по времени для производительности и управления ретенцией.
- **Денормализованный снимок.** vehicle_state хранит last-known состояние каждого ТС для мгновенной отрисовки карты на 1000+ ТС.

7.2. Описание ключевых сущностей

Сущность	Назначение	Ключевые поля
vehicle_type	Справочник типов техники	code, category, powertrain, is_autonomous
vehicle	Реестр машин	id (UUID), external_id, vehicle_type_id, status, attributes(JSONB)
sensor_type	Каталог измеряемых параметров	code, unit, value_type, min/max, category
vehicle_sensor	Датчики конкретного ТС	vehicle_id, sensor_type_id, calibration(JSONB)
metric	Временной ряд телеметрии	time, vehicle_id, sensor_type_id, value_num/bool/text/json, quality
event_type	Справочник типов событий	code, category, default_severity

Сущность	Назначение	Ключевые поля
event	Зафиксированные события/нарушения	id, time, vehicle_id, event_type_id, severity, payload(JSONB)
vehicle_state	Текущее состояние (снимок)	vehicle_id, last_seen, status, last_lat/lon, last_metrics(JSONB)
mission	Миссии ВАТС	id, vehicle_id, status, planned_route(JSONB), progress_pct
geofence	Геозоны	id, kind, geom(Polygon), is_active
alert_rule	Правила детекции и оповещений	condition(JSONB), severity, actions(JSONB)
app_user / app_role / user_role	Пользователи и RBAC	login, roles

7.3. ER-диаграмма

ER-диаграмма универсальной модели данных

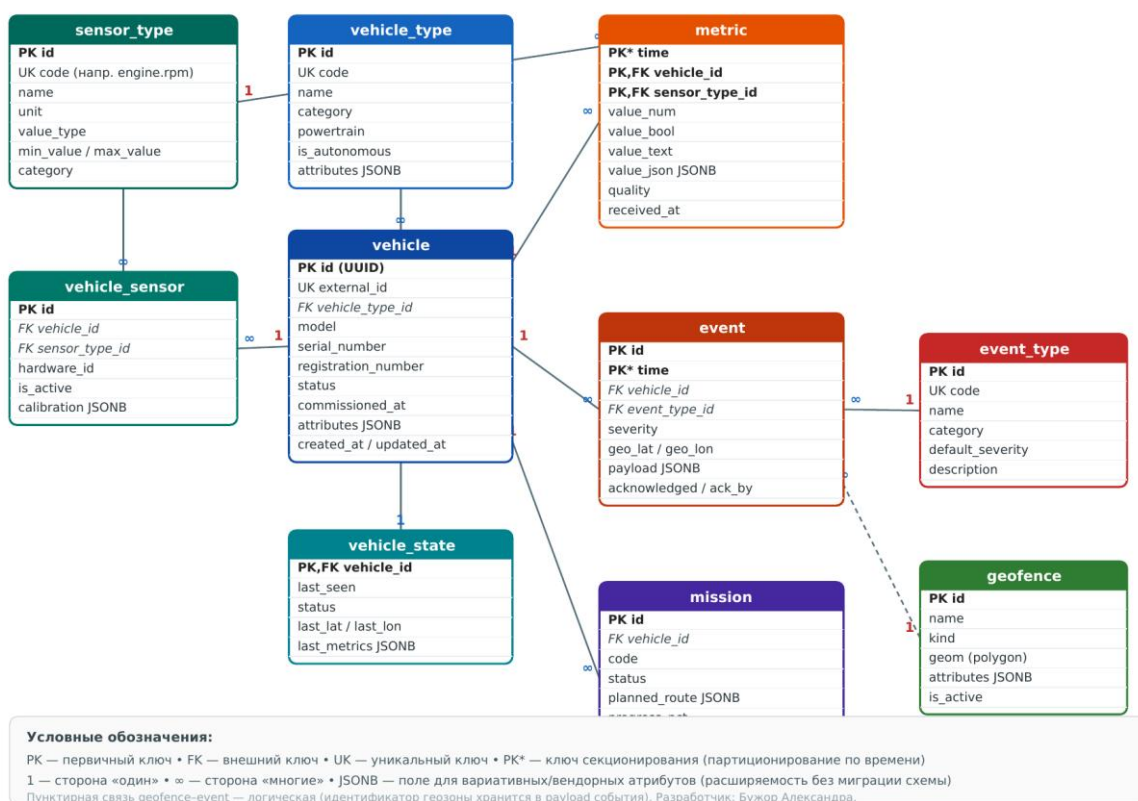


Рис. 2. ER-диаграмма универсальной модели данных.

7.4. Партиционирование и индексы

- **metric**: PARTITION BY RANGE(time) (помесячно/подекадно) или hypertable TimescaleDB (чанки по 1 дню). Индексы: (vehicle_id, sensor_type_id, time DESC) и BRIN по time.
- **event**: PARTITION BY RANGE(time) помесячно. Индексы: (vehicle_id, time DESC), (event_type_id, time), частичный индекс по открытым событиям, GIN по payload.
- **vehicle**: GIN-индекс по attributes для поиска по вариативным полям.
- **vehicle_state**: GIST-индекс по geom для запросов «в пределах области» (bbox/радиус).

Полный DDL со всеми таблицами, ограничениями, индексами и примерами заполнения справочников вынесен в Приложение Б (файл schema.sql).

Фрагмент DDL: таблица метрик

```
CREATE TABLE metric (
  time          TIMESTAMPTZ NOT NULL,
  vehicle_id    UUID        NOT NULL,
```

```

sensor_type_id INT NOT NULL,
value_num DOUBLE PRECISION,
value_bool BOOLEAN,
value_text TEXT,
value_json JSONB, -- geo/составные значения
quality SMALLINT NOT NULL DEFAULT 1,
received_at TIMESTAMPTZ NOT NULL DEFAULT now()
) PARTITION BY RANGE (time);

CREATE INDEX ix_metric_vst ON metric (vehicle_id, sensor_type_id, time DESC);
CREATE INDEX ix_metric_brin ON metric USING BRIN (time);

```

8. Спецификация API

8.1. Общие принципы

- Единый REST API для всех типов ТС; базовый путь /api/v1; версионирование в пути.
- Форматы — JSON (UTF-8); геоданные — GeoJSON; время — ISO 8601 в UTC.
- Аутентификация: пользователи — Bearer JWT (OAuth2) + RBAC; устройства — mTLS/токен (X-Device-Token).
- Пагинация (page/page_size), фильтрация и сортировка через query-параметры; лимиты запросов (rate limiting).
- Единый формат ошибок (code, message, trace_id, details); коды 400/401/403/404/409/429/5xx.

Полная машиночитаемая спецификация в формате OpenAPI 3.0.3 — Приложение А (файл openapi.yaml). Её можно открыть в Swagger UISwagger Editor и сгенерировать клиентов/серверные заглушки.

8.2. Основные методы

Метод / путь	Назначение
POST /ingest/telemetry	Приём батча метрик от устройства
POST /ingest/events	Приём событий, детектированных на устройстве
GET /fleet/state	Текущее состояние всего парка (для карты)
GET /vehicles	Список ТС (фильтры, пагинация)
POST /vehicles	Регистрация ТС
GET /vehicles/{id}	Карточка ТС
GET /vehicles/{id}/metrics	Временной ряд телеметрии (from/to, агрегация)
GET /vehicles/{id}/metrics/latest	Последние значения датчиков
GET /events	Реестр событий/нарушений
POST /events/{id}/ack	Квитирование события
GET/POST /missions, PATCH /missions/{id}	Управление миссиями BATC
GET/POST /geofences, /alert-rules	Геозоны и правила алертов
GET /reports/fleet-efficiency	Отчёт по эффективности парка
GET /catalogs/*	Справочники (типы ТС/датчиков/событий)

8.3. Форматы входных / выходных данных

Вход: батч телеметрии (POST /ingest/telemetry)

```

{
  "vehicle_id": "FRK-014",
  "device_id": "TBU-000771",
  "sent_at": "2026-06-30T10:14:05Z",
  "seq": 84213,
  "metrics": [
    {
      "sensor": "gps.location", "ts": "2026-06-30T10:14:03Z",
      "value": { "lat": 55.751, "lon": 37.618, "speed": 0, "sats": 11 } },
    { "sensor": "battery.soc", "ts": "2026-06-30T10:14:03Z", "value": 47, "unit": "%", "q": 1 },
    { "sensor": "battery.temp", "ts": "2026-06-30T10:14:03Z", "value": 61, "unit": "°C", "q": 1 }
  ]
}

```

Выход: временной ряд (GET /vehicles/FRK-014/metrics)

```
{
  "vehicle_id": "FRK-014", "from": "...", "to": "...", "agg": "avg", "interval": "PT1M",
  "series": [
    { "sensor": "battery.temp", "unit": "°C",
      "points": [ { "ts": "2026-06-30T10:00:00Z", "value": 54.2 },
                  { "ts": "2026-06-30T10:01:00Z", "value": 55.8 } ] }
  ]
}
```

Выход: состояние парка (GET /fleet/state)

```
{
  "as_of": "2026-06-30T10:14:06Z",
  "vehicles": [
    { "vehicle_id": "FRK-014", "type": "forklift", "powertrain": "ev",
      "status": "alarm", "position": { "lat": 55.751, "lon": 37.618, "speed": 0 },
      "key_metrics": { "battery.soc": 47, "battery.temp": 61 }, "active_alarms": 1 }
  ]
}
```

9. Прототипы пользовательских интерфейсов (Wireframes)

Приведены wireframes (мид-фиделити) четырёх ключевых экранов. Они определяют состав и компоновку элементов и служат основой для UX/UI-дизайна; финальный визуальный стиль уточняется на этапе дизайна.

9.1. Мониторинг парка — карта в реальном времени

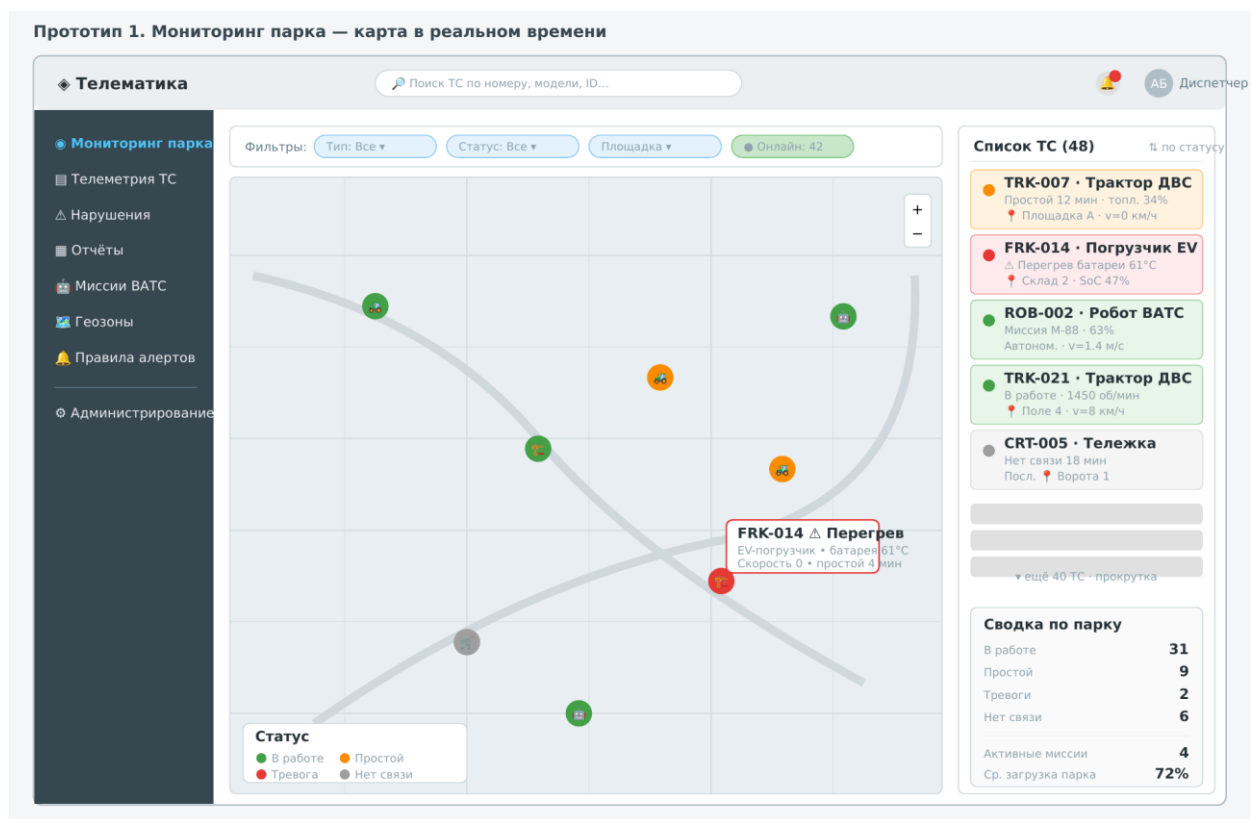


Рис. 3. Экран мониторинга: карта со статусами ТС, фильтры, список машин, сводка по парку.

Слева — навигация; в центре — карта с маркерами ТС (цвет = статус); справа — список ТС и сводка. Обновление статусов — в реальном времени. Поддерживает сценарии US-1, US-4.

9.2. Карточка ТС — телеметрия в реальном времени

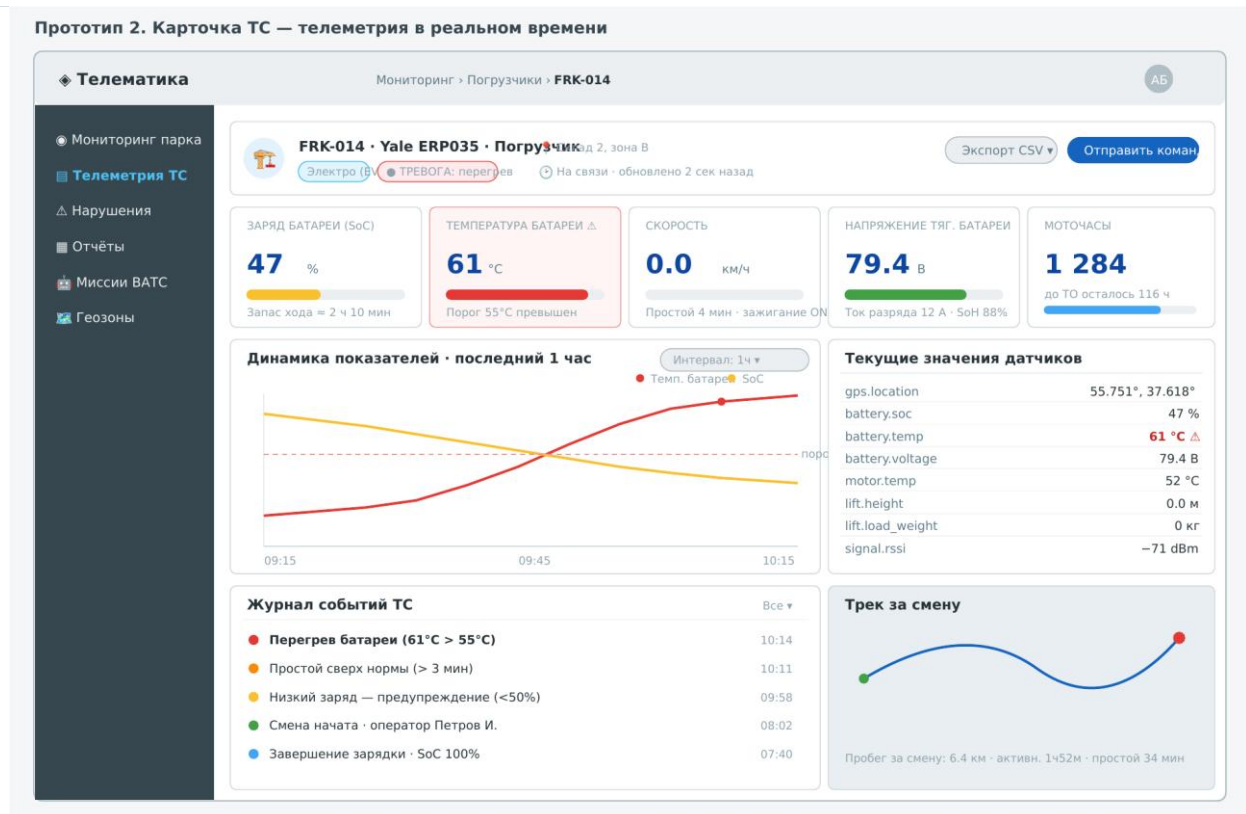


Рис. 4. Карточка ТС: текущие значения, графики, журнал событий, трек за смену.

Плитки текущих значений с индикацией порогов, графики динамики, список датчиков, журнал событий и мини-трек. Универсальна: набор плиток зависит от типа ТС (для EV — заряд/температура батареи, для ДВС — обороты/топливо). Сценарий US-2.

9.3. Аналитика нарушений — дашборд

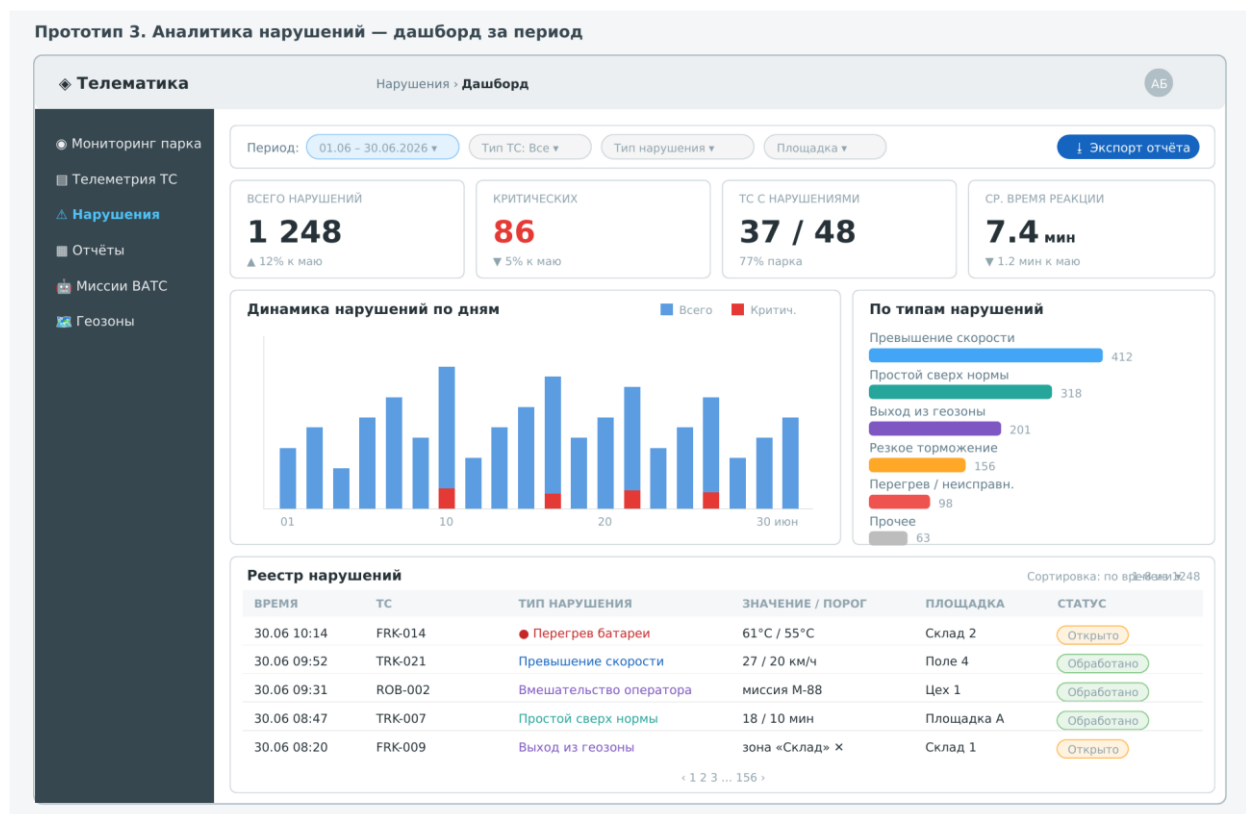


Рис. 5. Дашборд нарушений: KPI, динамика по дням, разбивка по типам, реестр.

Фильтры по периоду, типу ТС, типу нарушения и площадке; KPI; диаграммы динамики и структуры; реестр со статусами обработки. Сценарий US-3.

9.4. Отчёт по эффективности + сравнение ТС

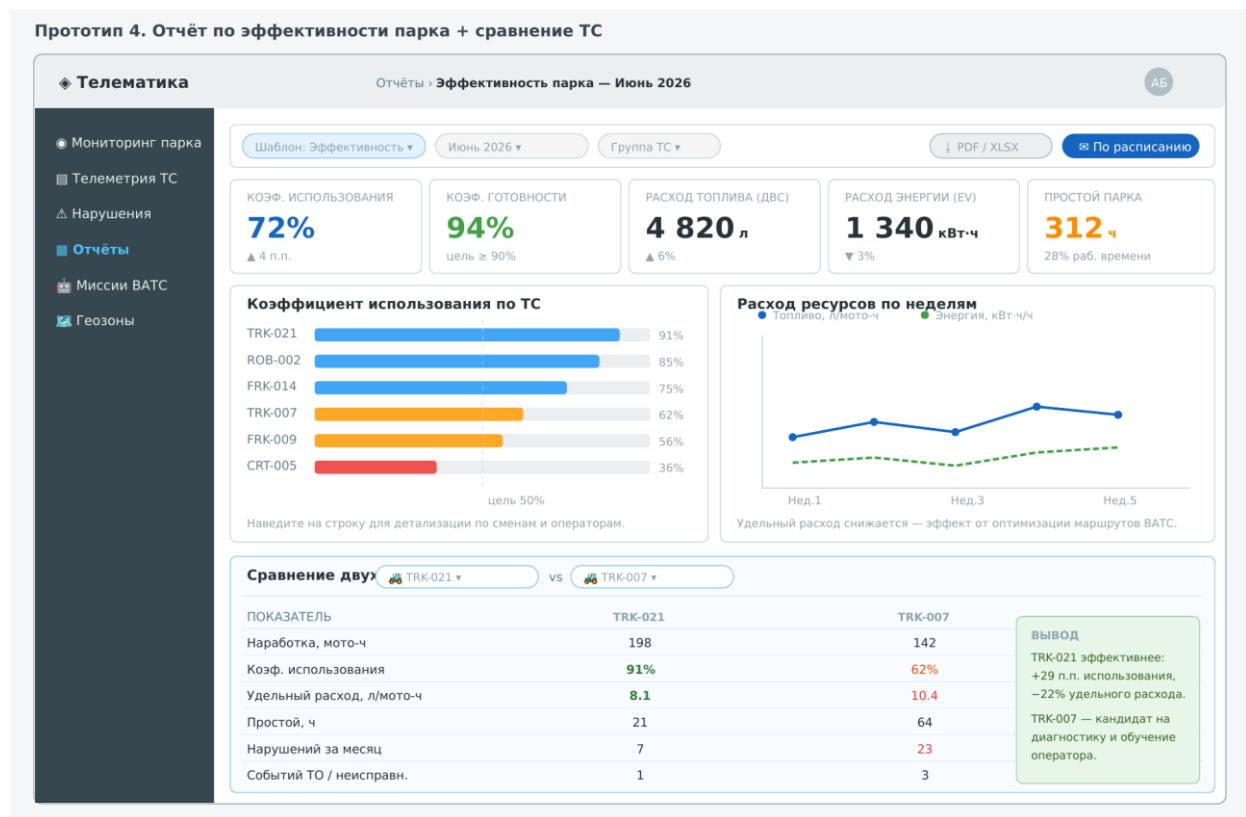


Рис. 6. Отчёт по эффективности парка и блок сравнения двух ТС.

KPI парка, использование по ТС, динамика расхода ресурсов, экспорт (PDF/XLSX) и расписание рассылки; блок сравнения двух ТС с автоматическим выводом. Сценарии US-5, US-7.

10. Требования к инфраструктуре и DevOps

10.1. Развёртывание

- Контейнеризация сервисов (Docker); оркестрация Kubernetes; окружения dev/staging/prod.
- Stateless-сервисы приёма/обработки/API масштабируются горизонтально; состояние — в Kafka/БД/Redis.
- Управляемые/выделенные кластеры: PostgreSQL+TimescaleDB (реплика), Kafka, Redis, объектное хранилище S3/MinIO.

10.2. CI/CD

- Пайплайны CI: сборка, статический анализ, тесты (unit/contract по OpenAPI), сборка образов.
- CD: автоматический деплой в staging, ручное подтверждение в prod; миграции БД версионизируются (Flyway/Liquibase).

10.3. Наблюдаемость и эксплуатация

- Метрики (Prometheus) и дашборды (Grafana): пропускная способность приёма, лаги Kafka, задержки API, здоровье БД.
- Централизованные логи и распределённая трассировка; алерты по SLO платформы.
- Резервное копирование БД и объектного хранилища; регулярные проверки восстановления; план аварийного восстановления (RPO/RTO по согласованию).

Компонент	Технология (предложение)	Роль
Приём	EMQX/VerneMQ (MQTT), сервис HTTP Gateway	Приём и аутентификация устройств
Шина	Apache Kafka	Буфер и разделение потоков

Компонент	Технология (предложение)	Роль
Обработка	Kafka Streams / Apache Flink	Валидация, детекция событий, агрегация
Хранение BP	TimescaleDB (PostgreSQL)	Метрики (партиции)
Хранение реестра	PostgreSQL + PostGIS	ТС, события, миссии, геозоны
Кэш	Redis	Текущее состояние парка
Архив	S3 / MinIO (Parquet)	Холодный слой
API	Сервис REST (Node.js/Java/Go) + WebSocket	Выдача и real-time
Оркестрация	Kubernetes + Docker	Развёртывание и масштабирование

11. Этапы реализации

Этап	Содержание	Результат
Э1. Фундамент	Модель данных, приём (MQTT/HTTP), хранение метрик, реестр ТС.	Приём и хранение телеметрии, БД по схеме
Э2. Мониторинг (MVP)	Карта парка, карточка ТС, состояние в реальном времени, базовые события.	Рабочий мониторинг для диспетчера
Э3. Аналитика	Правила алертов, оповещения, дашборд нарушений, отчёты.	Аналитика и оповещения
Э4. ВАС	Миссии, режимы автономности, события безопасности ВАС.	Поддержка автономной техники
Э5. Интеграции и масштаб	Публичный API, сравнение ТС, нагрузочное тестирование до 1000+ ТС, холодный слой.	Готовность к промышленной эксплуатации

12. Ожидаемые результаты и приложения

Настоящий пакет закрывает все ожидаемые результаты задания:

- Документ с пользовательскими историями и функциональными требованиями — разделы 3–5.
- Прототипы интерфейсов (wireframes) — раздел 9 (рис. 3–6).
- Описание архитектуры сбора и хранения данных — раздел 6 (рис. 1).
- ER-диаграмма модели данных — раздел 7 (рис. 2).
- Спецификация API (OpenAPI) — **Приложение А: openapi.yaml**.
- Спецификация форматов входных/выходных данных — раздел 8.3.
- SQL-схема БД (DDL) — **Приложение Б: schema.sql**.
- Итоговое ТЗ для разработчиков — настоящий документ целиком.

— Конец документа —

Подготовила: Бужор Александра · Версия 1.0 · 08.07.2026